



CHƯƠNG 3

Điều phối hệ thống với Docker Com- pose

Triển khai và quản trị hệ thống phần
mềm



Giới thiệu nội dung

Mục tiêu chương

- Giải thích vai trò và thành phần chính của Docker Compose.
- Viết cú pháp `docker-compose.yml` và dùng thành thạo các lệnh cơ bản.
- So sánh Docker Compose với Kubernetes; triển khai hệ đa dịch vụ WordPress/MySQL/Redis.

Nội dung chính

- 1** Vai trò và thành phần Compose
- 2** Cú pháp và các lệnh thường dùng
- 3** So sánh Compose và Kubernetes
- 4** Thực hành: triển khai WordPress

Vì sao cần điều phối nhiều container cùng lúc?

- Ứng dụng microservice thực tế hiếm khi chỉ có một container: ví dụ cần đồng thời container Node.js, container MongoDB và container Redis.
- Chạy từng container riêng lẻ bằng tay đòi hỏi gõ nhiều lệnh, tự thiết lập network và volume cho từng container – **rất phức tạp và dễ sai sót**.
- Cần một cách khai báo **toàn bộ hệ thống dịch vụ** trong một chỗ, rồi khởi động tất cả chỉ bằng một lệnh.

Câu hỏi mở đầu

Nếu hệ thống của ta gồm 5 container liên quan đến nhau, mỗi lần thay đổi cấu hình có cần gõ lại 5 lệnh `docker run` không? Có cách nào tốt hơn?

Docker Compose là gì?

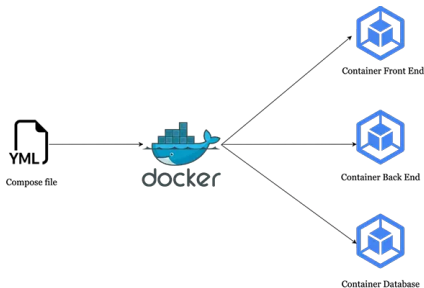
- Công cụ định nghĩa và chạy ứng dụng **đa container (multi-container)** bằng một file cấu hình duy nhất.
- Đóng vai trò như một “nhạc trưởng” điều phối các container hoạt động cùng nhau.
- Quản trị viên dùng file YAML để cấu hình services, sau đó dùng lệnh để tạo và thực thi cấu hình đó.

Ba bước cơ bản

- 1 Khai báo môi trường hoạt động của ứng dụng trong `Dockerfile`.
- 2 Khai báo các dịch vụ cần thiết trong `docker-compose.yml`.
- 3 Thực thi `docker-compose up` để khởi tạo và chạy toàn bộ dịch vụ.

Mô hình hoạt động của Docker Compose

- Từ tệp cấu hình YAML, Docker engine tạo và chạy container tương ứng cho mỗi dịch vụ.
- Các container trao đổi dữ liệu với nhau trong một **network khép kín**, cô lập với môi trường bên ngoài.
- Không chịu ảnh hưởng từ điều kiện máy host – tiết kiệm thời gian và đảm bảo nhất quán giữa các môi trường.

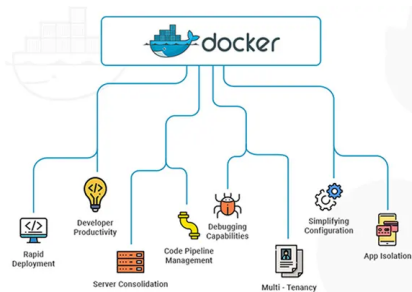


Các thành phần chính của Docker Compose

- **Compose file:** tệp YAML (`docker-compose.yml`) chứa toàn bộ thông tin để triển khai ứng dụng.
- **Services & Container:** mỗi service đại diện một thành phần ứng dụng (backend, frontend, database...); Compose dùng container để triển khai từng service.
- **Networks:** mạng ảo giúp các container liên lạc với nhau bằng tên hoặc địa chỉ IP.
- **Volumes:** lưu trữ dữ liệu độc lập với container, giúp sao lưu và phục hồi dễ dàng.
- **Environment variables & CLI:** biến môi trường cấu hình linh hoạt cho từng dịch vụ; CLI dùng để tạo, triển khai, quản lý và xóa container/service.

Lợi ích khi dùng Docker Compose

- **Đơn giản hóa quy trình:** một lệnh `docker-compose up` thay cho hàng loạt lệnh phức tạp.
- **Nhất quán giữa các môi trường:** giảm lỗi “chạy được trên máy tôi nhưng không chạy trên server”.
- **Hỗ trợ làm việc nhóm:** chia sẻ file YAML để đồng nghiệp tái tạo toàn bộ hệ thống.
- **Tích hợp microservices hiệu quả:** chạy đồng thời nhiều dịch vụ, kết nối mạng được thiết lập tự động.



Hạn chế cần lưu ý

Giới hạn của Docker Compose

- **Không phù hợp quy mô lớn:** thiết kế cho phát triển và triển khai nhỏ đến trung bình, không dành cho hàng trăm container cần tự động mở rộng.
- **Tiêu tốn tài nguyên:** chạy nhiều container cùng lúc vẫn cần đủ RAM/CPU trên máy chủ.
- **Hạn chế bảo mật:** file `docker-compose.yml` thường chứa mật khẩu, API key – dễ lộ nếu chia sẻ không cẩn thận.
- **Thiếu tính năng quản lý nâng cao:** không có tự phục hồi (self-healing), cân bằng tải hay giám sát chi tiết như Kubernetes.

Khi nào nên dùng Docker Compose?

Phù hợp khi

- Giai đoạn phát triển: tạo môi trường local nhanh để thử nghiệm.
- Dự án nhỏ và vừa: số lượng container hạn chế.
- Kiểm thử: mô phỏng môi trường sản xuất để chạy test tự động.

Nên cân nhắc thay thế khi

- Yêu cầu khả năng mở rộng lớn.
- Cần tính sẵn sàng cao (high availability).
- Đã hoàn thiện giai đoạn phát triển, chuẩn bị mở rộng sản xuất.

Cấu trúc cơ bản của file docker-compose.yml

```
services:
  web:
    build: .
    ports:
      - "${APP_PORT}:5000"
    environment:
      - REDIS_HOST=${REDIS_HOST}
      - REDIS_PORT=${REDIS_PORT}

  redis:
    image: redis:alpine
```

- Dịch vụ **web** build image từ Dockerfile trong thư mục hiện hành, ánh xạ cổng và nhận biến môi trường.
- Dịch vụ **redis** dùng thẳng image có sẵn từ Docker Hub, không cần build.

Lệnh nền tảng: khởi động và dừng dịch vụ

Lệnh	Mô tả
<code>docker compose up</code>	Khởi động tất cả service
<code>docker compose up -d</code>	Khởi động ở chế độ nền (khuyến nghị)
<code>docker compose up --build</code>	Build lại image rồi khởi động
<code>docker compose down</code>	Dừng và xóa container
<code>docker compose down -v</code>	Dừng và xóa cả volume
<code>docker compose restart</code>	Khởi động lại tất cả service

Lệnh vận hành: build, scale, theo dõi, tương tác

Lệnh	Mô tả
<code>docker compose build</code>	Build tất cả image
<code>docker compose up -d --scale web=3</code>	Scale service <code>web</code> lên 3 instance
<code>docker compose ps</code>	Xem danh sách container đang chạy
<code>docker compose logs -f</code>	Xem log realtime (follow)
<code>docker compose exec web sh</code>	Vào terminal của service <code>web</code>
<code>docker compose config</code>	Kiểm tra file cấu hình có lỗi không

Kết hợp lệnh trong một tình huống debug thực tế

Khởi động lại một service cụ thể

```
docker compose restart web
```

Xem log + theo dõi realtime

```
docker compose logs -f web
```

Vào container để debug

```
docker compose exec web sh
```

Build lại và khởi động lại service

```
docker compose up -d --build web
```

Điểm cốt lõi

Các lệnh Compose có thể phối hợp với nhau theo đúng thứ tự vấn đề cần xử lý: quan sát log, vào container kiểm tra, rồi build lại nếu cần.

Kubernetes: điều phối container ở quy mô cluster

- **Kubernetes (K8s)** là hệ thống mã nguồn mở tự động hóa triển khai, nhân rộng và quản lý ứng dụng container trên **hiều máy chủ (cluster)**.
- Tên gọi bắt nguồn từ tiếng Hy Lạp, nghĩa là “người lái tàu/hoa tiêu”; Google mở mã nguồn từ năm 2014.
- Trong môi trường production, cần đảm bảo hệ thống **không downtime**: khi một container bị lỗi, một container khác phải khởi động thay thế – đây là bài toán Kubernetes giải quyết.

Câu hỏi bắc cầu

Docker Compose hạn chế ở quy mô lớn và không tự phục hồi – vậy công cụ nào giải quyết được đúng những hạn chế đó?

Kubernetes cung cấp gì mà Compose không có?

- **Service discovery & cân bằng tải:** expose container qua DNS/IP riêng, tự động phân phối traffic.
- **Tự động rollouts & rollbacks:** chuyển trạng thái thực tế sang trạng thái mong muốn với tần suất kiểm soát được.
- **Tự phục hồi (self-healing):** tự khởi động lại container lỗi, thay thế container không đạt health check.
- **Đóng gói tự động:** điều phối container vào các node dựa trên yêu cầu CPU/RAM để tận dụng tài nguyên.
- **Quản lý cấu hình & bảo mật:** lưu trữ secret (password, token, SSH key) và cập nhật mà không cần build lại image.

Bảng so sánh Docker Compose và Kubernetes

Tiêu chí	Docker Compose	Kubernetes
Mục đích chính	Đa container trên một máy	Orchestration trên nhiều máy (cluster)
Độ phức tạp	Rất đơn giản, dễ học	Rất phức tạp, dốc học cao
Scaling	Thủ công (<code>--scale</code>)	Tự động (Horizontal Pod Autoscaler)
High Availability	Không hỗ trợ tốt (1 máy)	Hỗ trợ mạnh: self-healing, replica, failover
Secret & Config	<code>.env</code> và <code>environment</code>	ConfigMap + Secret (an toàn, chuyên nghiệp)
Learning curve	Dễ, phù hợp beginner	Khó, cần thời gian học sâu

Chọn Docker Compose hay Kubernetes?

Docker Compose

- Phát triển local, test, demo.
- Dự án nhỏ, dưới 10 services.
- Production nhỏ (1–2 server).
- Startup giai đoạn đầu.

Kubernetes

- Production trung bình và lớn.
- Cần scale tự động, high availability.
- Team có DevOps mạnh.
- Doanh nghiệp lớn, hệ thống phân tán.

Bài toán thực hành: hệ thống WordPress đa dịch vụ

- Triển khai CMS mã nguồn mở **WordPress** (frontend + PHP), **MySQL** (cơ sở dữ liệu) và **Redis** (cache) bằng Docker Compose.
- Tất cả dịch vụ chạy trong một network độc lập: `wp-network`.

```
wordpress-docker-lab/  
├── docker-compose.yml  
├── .env  
├── Dockerfile           # Dockerfile cho WordPress (custom image)  
├── init.sql             # (tùy chọn) script khởi tạo DB  
└── README.md
```

Áp dụng

Đây là nơi toàn bộ lý thuyết về services, networks, volumes và environment variables được ráp lại thành một hệ thống thật.

Biến môi trường: file .env

```
# MySQL
MYSQL_ROOT_PASSWORD=SuperSecretRootPass123
MYSQL_DATABASE=wordpress
MYSQL_USER=wp_user
MYSQL_PASSWORD=wp_password_strong_456

# WordPress
WORDPRESS_DB_HOST=db
WORDPRESS_DB_USER=wp_user
WORDPRESS_DB_PASSWORD=wp_password_strong_456
WORDPRESS_DB_NAME=wordpress

# Redis (cache)
REDIS_PASSWORD=redis_secure_pass_789
```

Bảo mật

File `.env` chứa thông tin nhạy cảm – **không commit lên Git**. Thường dùng file `.env` riêng cho từng môi trường (dev, stage, production).

Dockerfile: build custom image cho WordPress

```
# Sử dụng image chính thức của WordPress làm base
FROM wordpress:6.6.1-php8.3-apache

# Cài đặt Memcached extension (tùy chọn)
RUN apt-get update && apt-get install -y --no-install-recommends \
    libmemcached-dev zlib1g-dev libssl-dev \
    && pecl install memcached \
    && docker-php-ext-enable memcached \
    && apt-get purge -y --auto-remove zlib1g-dev libssl-dev \
    && apt-get clean && rm -rf /var/lib/apt/lists/* /tmp/pear/*

# Đặt quyền thư mục
RUN chown -R www-data:www-data /var/www/html

EXPOSE 80
```

docker-compose.yml: dịch vụ db và redis

```
services:
  db:
    image: mysql:8.0
    env_file: [.env]
    environment:
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
      MYSQL_DATABASE: ${MYSQL_DATABASE}
    volumes:
      - db_data:/var/lib/mysql
    networks: [wp-network]
    healthcheck:
      test: ["CMD-SHELL", "mysqladmin ping -p${MYSQL_ROOT_PASSWORD}"]
      interval: 10s

  redis:
    image: redis:7.2-alpine
    command: redis-server --requirepass ${REDIS_PASSWORD}
    volumes:
      - redis_data:/data
    networks: [wp-network]
```

docker-compose.yml: dịch vụ wordpress, network và volume

```
wordpress:
  build: .
  depends_on:
    db:
      condition: service_healthy
    redis:
      condition: service_started
  ports: ["8080:80"]
  environment:
    WORDPRESS_DB_HOST: db
    WORDPRESS_DB_USER: ${MYSQL_USER}
    WORDPRESS_DB_PASSWORD: ${MYSQL_PASSWORD}
  volumes:
    - wp_data:/var/www/html
  networks: [wp-network]

networks:
  wp-network:
    driver: bridge
```

Vì sao cần `depends_on` kết hợp `healthcheck`?

- `depends_on` đảm bảo **thứ tự khởi động**: WordPress chỉ khởi động sau khi `db` và `redis` đã bắt đầu.
- Riêng `db` dùng điều kiện `condition: service_healthy` – WordPress phải chờ MySQL **thực sự sẵn sàng nhận kết nối**, không chỉ mới khởi động tiến trình.
- `restart: unless-stopped` giúp dịch vụ tự khởi động lại khi gặp sự cố.

Điểm dừng: điều gì xảy ra nếu bỏ `healthcheck`?

Nếu bỏ `healthcheck` cho `db`, `depends_on` chỉ biết container MySQL đã *khởi động tiến trình*, chưa chắc đã sẵn sàng nhận kết nối. WordPress có thể cố kết nối DB quá sớm và lỗi ngay lần chạy đầu tiên.

Triển khai: build và khởi động hệ thống

Build image và khởi động toàn bộ hệ thống ở chế độ nền

```
docker compose up -d --build
```

Kiểm tra các container đã được khởi tạo đầy đủ chưa

```
docker compose ps
```

Theo dõi log nếu service wordpress gặp lỗi

```
docker compose logs -f wordpress
```

Truy cập ứng dụng

Sau khi container khởi động xong, mở trình duyệt tại **http://localhost:8080** và hoàn tất cài đặt WordPress (ngôn ngữ, tài khoản admin).

Kết quả: hệ thống khởi động thành công

```
=> => unpacking to docker.io/library/wp-docker-wordpress:latest
=> resolving provenance for metadata file
[+] up 6/6
♦ Image wp-docker-wordpress Built
♦ Network wp-network Created
♦ Container wp-redis Started
♦ Container wp-mysql Started
♦ Container wp-phpmyadmin Started
♦ Container wp-app Started
```

Các container được build và start, network được khởi tạo

```
Container wp-app Started
anhchuyen@linuxserver:~/wp-docker$ docker compose ps
WARN[0000] /home/anhchuyen/wp-docker/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
NAME        IMAGE                COMMAND                SERVICE    CREATED     STATUS      PORTS
wp-app      wp-docker-wordpress  "docker-entrypoint.s..." wordpress  7 seconds ago Up 6 seconds 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
wp-mysql    mysql:8.0            "docker-entrypoint.s..." db         7 seconds ago Up 6 seconds 3306/tcp, 33060/tcp
wp-phpmyadmin phpmyadmin:latest    "/docker-entrypoint.s..." phpmyadmin 7 seconds ago Up 6 seconds 0.0.0.0:8081->80/tcp, [::]:8081->80/tcp
wp-redis    redis:7.2-alpine     "docker-entrypoint.s..." redis      7 seconds ago Up 6 seconds 6379/tcp
anhchuyen@linuxserver:~/wp-docker$
```

Danh sách container đang hoạt động (docker compose ps)

Dùng và dọn dẹp hệ thống

- Lệnh `docker compose down` loại bỏ toàn bộ container và network đã tạo cho hệ thống.
- Thêm cờ `-v` nếu muốn xóa luôn cả volume dữ liệu.
- Dữ liệu trong volume (`db_data`, `wp_data`) vẫn tồn tại nếu không dùng `-v` – có thể khởi động lại hệ thống mà không mất dữ liệu.

```
anhchuyen@linuxserver: ~/wp-docker$ docker compose down
WARN[0000] /home/anhchuyen/wp-docker/docker-compose.yml: the attr
[+] down 5/5
✔ Container wp-phpmyadmin Removed
✔ Container wp-app Removed
✔ Container wp-mysql Removed
✔ Container wp-redis Removed
✔ Network wp-network Removed
```

Tổng kết chương

Những điểm cần ghi nhớ

- Docker Compose điều phối nhiều container bằng một file YAML duy nhất, gồm services, networks, volumes, environment variables.
- Đơn giản hóa triển khai đa dịch vụ nhưng hạn chế về mở rộng, tự phục hồi và bảo mật thông tin nhạy cảm.
- Kubernetes điều phối container ở quy mô cluster (service discovery, tự phục hồi, rollout/rollback); lựa chọn công cụ phụ thuộc quy mô và năng lực đội ngũ.
- Đã triển khai WordPress + MySQL + Redis trên `wp-network`, dùng `depends_on` kết hợp `healthcheck` để kiểm soát thứ tự khởi động.